

Tarea 1.

1. Entrada:

Un arreglo $A[0..N]$, $N \geq 0$ de números enteros.

Salida:

Un arreglo ordenado B que se construye con $A[0..N]$ y un número entero n que representa el total de inversiones para ordenar el arreglo $A[0..N]$

Nota: Una inversión es un par de índices (i, j) tal que $0 \leq i < j < N$ y $A[i] > A[j]$

Ej: Para $A = [3, 2, 4, 1]$, las inversiones son:

$(3, 2), (3, 1), (2, 1), (4, 1)$ Para un total de 4.

Planteamiento función Objetivo:

Sea $0 \leq l < h \leq N$

$\phi(l, h)$: Determina el número de inversiones en $A[l..h]$ y deja ordenado dicho arreglo

Reformulación de la especificación:

$\phi(0, N)$

Planteamiento Recursivo:

$$\phi \begin{cases} 0 & \text{if } h - l \leq 1 \\ \phi(l, \text{mid}) + \phi(\text{mid}, h) + \mu(l, \text{mid}, h) & \text{if } l + 1 < h \end{cases}$$

Nota: sea mid:
 $(l + h) // 2$

Nota 2: sea μ la función que recibe 2 mitades ordenadas y las combina para ordenar ambas mitades

código:

```
6 def merge(arr, low, mid, high):
7     tmp = [0] * len(arr)
8     # make a copy of current segment
9     for i in range(low, high):
10         tmp[i] = arr[i]
11     # iterators for tmp
12     l, r = low, mid
13     inv = 0
14     # n will iterate on arr
15     for n in range(low, high):
16         # l has reached the end of the left half
17         if l == mid:
18             arr[n], r = tmp[r], r + 1
19         # r has reached the end of the right half
20         elif r == high:
21             arr[n], l = tmp[l], l + 1
22         else:
23             # choose left
24             if tmp[l] <= tmp[r]:
25                 arr[n], l = tmp[l], l + 1
26             # choose right
27             else:
28                 arr[n], r = tmp[r], r + 1
29             # count inversions
30             inv += mid - l
31     return inv
32
33 def mergeSort(arr, low, high):
34     if low + 1 < high:
35         mid = (low + high) // 2
36         # count inversions for left(l) and right(r)
37         linv = mergeSort(arr, low, mid)
38         rin = mergeSort(arr, mid, high)
39         # merge two sorted halves
40         inv = merge(arr, low, mid, high)
41         ans = linv + rin + inv
42     else: ans = 0
43     return ans
```

¿Por qué contar las inversiones en merge?

1. Diseño: Usando la división de merge sort:

Para un arreglo $A(l..h)$ se divide:

$A(l..mid)$ y $A(mid..h)$

las inversiones se pueden clasificar como:

1. Inversiones de la mitad izquierda: Ambos elementos a invertir se encuentran en $A(l..mid)$
2. Inversiones de la mitad derecha: Ambos elementos a invertir se encuentran en $A(mid..h)$
3. Inversiones cruzadas: Un elemento se encuentra en $A(l..mid)$ y el otro en $A(mid..h)$

Por lo tanto el total de inversiones I es:

$$I(A) = I_l + I_h + I_c$$

I_l = Inversiones de la izquierda.

I_h = Inversiones de la derecha.

I_c = Inversiones cruzadas.

En merge está la modificación necesaria porque es precisamente durante la mezcla de dos mitades ordenadas donde podemos identificar las inversiones cruzadas.

Si $A(l..mid)$ $\wedge$ $A(mid..h)$ están ordenados
Ej: $\underbrace{[2, 5, 8]}_{\text{izquierda}}$ $\underbrace{[1, 4, 7]}_{\text{derecha}}$

Durante merge supongamos encontrar:

$$A[h] < A[l]$$

como la mitad izquierda está ordenada, sabemos:

$$A[l] \leq A[l+1] \leq \dots \leq A[mid-1]$$

Entonces $s_i: A[i] < A[l]$,

también se cumple que:

$$A[i] < A[l+1] < \dots < A[mid-1]$$

Por lo tanto todos los elementos en la mitad izquierda forman una inversión con $A[i]$. Siendo $mid-1$ la cantidad de elementos que se suman a las inversiones.

¿Por qué no contar en otro lugar?

Porque antes de hacer merge se pueden contar las inversiones de cada lado sin contar las cruzadas, lo que no permite calcular todas las inversiones.

2. Demostración por inducción:

Caso base:

un segmento de tamaño 0 o 1 no tiene pares (i, j) con $i < j$ por lo tanto el número de inversiones es cero y está trivialmente ordenado

caso inductivo:

sea $mid = \lfloor (l+h)/2 \rfloor$ y teniendo que $l \leq mid < h$ como hipotesis inductiva entonces suponemos que vale para cualquier l', h' que cumplen $l \leq l' < h' \leq h$ y $h' - l' < n$.

$l_{inv} = \text{mergeSort}(arr, l, mid)$: calcula correctamente $\phi(l, mid)$: El número de inversiones que tenía $A[l..mid]$ antes de esta llamada y deja $A[l..mid]$ ordenado.

$r_{inv} = \text{mergeSort}(arr, mid, r)$: calcula correctamente $\phi(mid, h)$ de igual forma sobre $A[mid..h]$ y lo deja ordenado.

Ahora falta demostrar que $inv = \text{merge}(arr, l, mid, h)$ calcula correctamente $\text{Merge}(l, mid, h)$ y que por lo tanto $ans = l_{inv} + r_{inv} + inv = \phi(l, h)$, dejando $arr[l..h]$ ordenado.

a) Toda inversión (i, j) de $A[l..h]$ en su estado original pertenece a una de las siguientes:

- Ambas en (l, mid) : Cuenta $\phi(l, mid)$
- Ambas en (mid, h) : cuenta $\phi(mid, h)$
- $i \in (l, mid)$ y $j \in (mid, h)$: son los cruzados

lo que significa que $i < mid \leq j$, bastando con que i este en la mitad izquierda y j en la mitad derecha.

Por lo tanto

$T_{inv} + T_{inv} + T_{inv} = O(l, mid) + O(mid, r) + M(l, mid, h)$
y el arreglo $A(l..h)$ queda ordenado
ascendentemente.